# CareerPilot AI Technical Project Report

## File map, architecture, feature ownership, and mentor explanation guide

Prepared for project presentation | October 24, 2026

**Purpose of this report**

This document explains how CareerPilot AI is organized, which files implement each feature, how the user journey works, and what can be presented to a mentor as the technical contribution of the project.

## Executive Summary

CareerPilot AI is a Flask-based career development platform. It combines user authentication, profiles, resume analysis, AI career guidance, interview practice, roadmaps, job matching, skill-gap analysis, cover-letter generation, application tracking, a career chatbot, and an administrator control panel.

The backend is divided into route modules, AI modules, and a database layer. The frontend uses server-rendered Jinja templates with shared CSS and JavaScript. Groq is used as the external AI provider, while deterministic local logic and fallback responses keep core workflows available when an API request cannot be completed.

**One-sentence explanation for a mentor**

CareerPilot AI is a full-stack Flask career platform that converts a user profile and resume into a connected workflow: analysis, career guidance, skill-gap planning, interview practice, job matching, and application tracking.

## System Architecture

```
User request -> app.py -> routes/ -> database/ and ai_modules/ -> templates/ -> static assets
```

The application uses a practical MVC-style separation. Flask routes receive requests and enforce access control. Database helpers read and write MySQL data. AI helpers call Groq and provide fallback behavior. Jinja templates render HTML, while CSS and JavaScript control presentation and browser interactions.

## Core Project Files

- **`D:\AI Career Portal\app.py`** - Main Flask entry point. Registers all blueprints, configures the application secret key, sets the upload limit, defines the home page, and handles 404, 413, and 500 errors.
- **`D:\AI Career Portal\config.py`** - Loads environment variables for MySQL, admin authentication, and the application configuration.
- **`D:\AI Career Portal\requirements.txt`** - Lists the Python dependencies used by the application, including Flask, Gunicorn, HTTPX, PDFPlumber, PyMySQL, and Python Dotenv.
- **`D:\AI Career Portal\.env`** - Local development configuration. Contains private values such as database credentials, the Groq key, the admin account, and the Flask secret key. This file must remain private.
- **`D:\AI Career Portal\.gitignore`** - Prevents .env, uploads, virtual-environment files, cache files, and generated files from being committed.
- **`D:\AI Career Portal\careerpilot_ai.sql`** - Master database export containing the main application tables and sample records.

## AI and Intelligence Layer

- **`D:\AI Career Portal\ai_modules\career_ai.py`** - Main Groq integration. Handles resume feedback, dashboard guidance, chatbot replies, interview questions, interview evaluation, and career roadmap generation. It checks GROQ_API_KEY and provides fallback behavior.

- **D:\AI Career Portal\ai_modules\career_tools_ai.py** - AI support for job matching, skill-gap insights, cover-letter drafting, and application-pipeline insights.
- **D:\AI Career Portal\ai_modules\resume_analyzer.py** - Local resume processor. Extracts PDF text, detects sections and skills, and calculates deterministic scores before AI feedback is added.
- **D:\AI Career Portal\ai_modules\usage.py** - AI governance and observability layer. Checks feature flags, records request timing, and logs success, failure, disabled, and missing-key results.

**Important deployment detail**

The live Railway service must have GROQ_API_KEY in its service variables. The local .env file is ignored by Git and is not automatically available to Railway. GROQ_MODEL is optional because the application has a default model.

## Database Layer

- **D:\AI Career Portal\database\db.py** - Creates MySQL connections, provides database sessions, creates or updates required tables at runtime, seeds the configured admin account, and ensures the AI-management schemas exist.
- **D:\AI Career Portal\database\auth_roles.sql** - Defines admin accounts, admin activity logs, AI feature flags, AI usage logs, and professional profile support.
- **D:\AI Career Portal\database\career_roadmaps.sql** - Defines persistent career-roadmap storage.
- **D:\AI Career Portal\database\career_tools.sql** - Defines the job-application tracker table.
- **D:\AI Career Portal\database\interview_results.sql** - Defines interview-session and score storage.

The main database entities are students, student profiles, professional profiles, admins, resume analyses, interview results, job applications, career roadmaps, AI feature flags, AI usage logs, and admin activity logs.

## Route and Feature Controllers

- **D:\AI Career Portal\routes\auth.py** - Student registration, login, logout, password hashing, fresher or experienced profile selection, and the login_required decorator.
- **D:\AI Career Portal\routes\admin.py** - Administrator login, dashboard, user directory, user status management, AI management, AI feature switches, activity logs, and admin password settings.
- **D:\AI Career Portal\routes\profile.py** - Profile editing, validation, profile completion, career direction fields, professional information, profile photos, and profile security checks.
- **D:\AI Career Portal\routes\dashboard.py** - Builds the fresher and experienced dashboards, calculates progress, loads user evidence, and requests AI dashboard guidance.
- **D:\AI Career Portal\routes\resume.py** - Handles resume upload, file validation, PDF extraction, resume analysis, analysis history, and resume-tailoring entry points.
- **D:\AI Career Portal\routes\interview.py** - Creates practice interviews, generates questions, evaluates answers, calculates results, and displays interview history.
- **D:\AI Career Portal\routes\roadmap.py** - Creates and displays a personalized roadmap for the user target role.
- **D:\AI Career Portal\routes\career_tools.py** - Implements the job matcher, skill-gap analyzer, cover-letter generator, application tracker, and resume-tailor workflow.
- **D:\AI Career Portal\routes\chatbot.py** - Loads student context, maintains chat history, sends questions to the AI layer, and clears the conversation.
- **D:\AI Career Portal\routes\about.py** - Renders the public About page.

# Feature to File Map

| Feature | Main files | What the feature does |
|---|---|---|
| **Authentication** | routes/auth.py; templates/auth/*; database/db.py | Creates accounts, signs users in, stores password hashes, and protects student-only pages. |
| **Profile** | routes/profile.py; templates/profile/*; database/db.py | Stores personal, academic, professional, preference, and career-direction information. |
| **Dashboard** | routes/dashboard.py; templates/dashboard/*; ai_modules/career_ai.py | Combines profile, resume, roadmap, and progress data into a personalized next-action view. |
| **Resume analysis** | routes/resume.py; ai_modules/resume_analyzer.py; ai_modules/career_ai.py; templates/resume/* | Extracts a PDF, calculates scores, identifies skills, and creates improvement feedback. |
| **Interview practice** | routes/interview.py; ai_modules/career_ai.py; templates/interview/*; database/interview_results.sql | Generates questions, evaluates answers, scores performance, and stores history. |
| **Career roadmap** | routes/roadmap.py; ai_modules/career_ai.py; templates/roadmap/index.html; database/career_roadmaps.sql | Turns a target role into a structured action plan. |
| **Job matching** | routes/career_tools.py; ai_modules/career_tools_ai.py; templates/tools/job_matcher.html | Compares user skills and preferences with a job description or target role. |
| **Skill gap analysis** | routes/career_tools.py; ai_modules/career_tools_ai.py; templates/tools/skill_gap.html | Shows existing skills, missing skills, and recommended next evidence. |
| **Cover letters** | routes/career_tools.py; ai_modules/career_tools_ai.py; templates/tools/cover_letter.html | Creates a tailored introduction using profile and job information. |
| **Application tracking** | routes/career_tools.py; templates/tools/application_tracker.html; database/career_tools.sql | Tracks job applications, statuses, priorities, dates, notes, and follow-up actions. |
| **Career chatbot** | routes/chatbot.py; ai_modules/career_ai.py; templates/chatbot/index.html | Provides conversational guidance using the user career context. |
| **Admin operations** | routes/admin.py; templates/admin/*; database/db.py | Provides user management, AI monitoring, feature switches, activity logs, and admin settings. |

# Templates and Frontend Assets

- **`D:\AI Career Portal\templates\layouts\base.html`** - Shared HTML layout. Loads the common navigation, footer, CSS, JavaScript, and page blocks.
- **`D:\AI Career Portal\templates\partials\navbar.html and footer.html`** - Shared site navigation, authentication links, admin links, and footer content.
- **`D:\AI Career Portal\templates\home\index.html and about\index.html`** - Public landing page and project information page.
- **`D:\AI Career Portal\templates\auth\*`** - Student registration and login screens.
- **`D:\AI Career Portal\templates\profile\* and dashboard\*`** - Student profile and fresher or experienced dashboard screens.
- **`D:\AI Career Portal\templates\resume\*`** - Resume upload, results, and history screens.
- **`D:\AI Career Portal\templates\interview\*`** - Interview setup, question, result, final-result, and history screens.
- **`D:\AI Career Portal\templates\roadmap\index.html`** - Roadmap creation, milestones, evidence, and action links.

- **`D:\AI Career Portal\templates\tools\*`** - Career tools pages plus shared tool navigation and AI insight components.
- **`D:\AI Career Portal\templates\chatbot\index.html`** - Career chatbot interface.
- **`D:\AI Career Portal\templates\admin\*`** - Admin login, dashboard, user management, AI management, settings, and activity pages.
- **`D:\AI Career Portal\static\css\style.css`** - Central visual design for all public, student, career-tool, interview, chatbot, roadmap, and admin pages.
- **`D:\AI Career Portal\static\js\script.js`** - Browser behavior such as loading states, resume upload feedback, chatbot actions, dashboard AI refresh, password visibility, validation, and profile photo preview.
- **`D:\AI Career Portal\static\images\hero.png`** - Hero visual used by the landing page.

## End-to-End User Journey

1. The user registers or logs in through routes/auth.py.
2. The user completes a profile through routes/profile.py.
3. The user uploads a resume through routes/resume.py.
4. resume_analyzer.py extracts text and calculates deterministic scores.
5. career_ai.py adds Groq-generated feedback when GROQ_API_KEY is available.
6. The user selects a target role and receives a roadmap through routes/roadmap.py.
7. The user checks job fit and missing skills through routes/career_tools.py.
8. The user creates applications, cover letters, and follow-up records.
9. The user practices interviews through routes/interview.py.
10. The dashboard combines progress, evidence, roadmaps, and AI guidance.
11. The administrator monitors users, AI usage, feature flags, and audit activity through routes/admin.py.

## Short Mentor Explanation

CareerPilot AI is a full-stack Flask career platform. The app uses blueprints to separate authentication, profiles, dashboards, resumes, interviews, roadmaps, career tools, chatbot functionality, and administration. MySQL stores user and career data. The AI layer uses Groq for personalized guidance but keeps deterministic local scoring and fallback behavior so the application can continue to serve users when the external provider is unavailable. The frontend uses Jinja templates, shared CSS, and JavaScript to provide a consistent user experience.

The main innovation is the connected workflow. Instead of offering isolated tools, CareerPilot AI connects profile data, resume evidence, target roles, skill gaps, learning roadmaps, interview performance, and job applications into one career workspace.

## Validation Completed

- Python syntax compilation passed for the application files.
- Dependency validation passed with no broken requirements.
- All 36 Jinja templates passed syntax parsing.
- The Flask application imported successfully and exposed 37 routes.
- Template endpoint references were checked and no missing url_for endpoints were found.
- Basic public route smoke tests passed for home, About, login, registration, admin login, chatbot, and tools entry points.
- No API-key-shaped secrets were found outside the ignored local .env file.

# Future Improvements to Mention

- Use formal database migrations instead of maintaining schema definitions in both database/db.py and separate SQL files.
- Add automated unit and integration tests; the current repository does not contain a dedicated test suite.
- Keep FLASK_DEBUG disabled in production and use a strong Railway SECRET_KEY.
- Add a dedicated deployment configuration file if the team wants deployment settings versioned with the repository.
- Add rate limiting, stronger CSRF coverage for every state-changing form, and clearer privacy controls for uploaded resumes.

**Final takeaway**

CareerPilot AI is not only an AI chatbot. It is a complete career workflow platform with user data, deterministic analysis, AI assistance, progress tracking, interview preparation, job matching, and administrator controls.